

```

0000 SEPARATE (IDL) ::= --
E DESCRIPTION IDL EN MEMOIRE VIRTUELLE% FILE_NAME: FILE_TYPE; FICHIER IDL% SLINE: STRING ( 1..256 ) IS% -- LIGNE-TEXTE COURANTE% COL:
NATURAL; NATURAL := 1; PROCHaine COLonne LIRE% F COL: NATURAL; PREMIERE COLonne DU LEXEME% TOKEN_LENGTH: NATURAL; LONGUE
UR DU LEXEME% LAST: NATURAL := 0; NOMBRE DE CARACTERE DE LA LIGNE% TOKEN_IS_NAME: BOOLEAN; LINE_COUNT: NATURAL := 0; ATTR_COUN
T: INTEGER := 1; SOURCE_LIST: SEQ_TYPE; LISTE DES LIGNES SOURCES% SOURCEPOS: TREE; LA POSITION SOURCE DU LEXEME COUR
SOURCELINE: TREE; LE NOEUD LIGNE SOURCE% USER_ROOT: TREE; RACINE DE L'ARBRE% TYPE CONTEXT_TYPE: IS (NIL, IN_NODE, IN_CL
ASS); CONTEXT: CONTEXT_TYPE := NIL; PROCEDURE PROCESS_IDL; PROCEDURE CHECK_IDL; PROCEDURE PRINT_IDL;
--PROCEDURE GET_TOKEN% PROCEDURE GET_TOKEN IS% BEGIN% WHILE COL < LAST AND
THEN ( SLINE (COL) = ' ' OR ELSE SLINE (COL) = ASCII.HT ) LOOP% PASSER LES ESPACES% COL := COL + 1; END LOOP; IF COL < LAST AND TH
EN SLINE (COL) = ' ' AND THEN SLINE (COL+1) = ' ' THEN% SAUTER EN FIN DE LIGNE SUR COMMENTAIRE% COL := LAST + 1; END IF; IF COL <
LAST THEN% SI ON-EST POST FIN DE LIGNE% LOOP% IF END_OF_FILE ( IFILE ) THEN RETURN; END IF; IF END_OF_FILE ( IFILE ) TH
EN% SKIP LINE ( IFILE ); SLINE_COUNT := LINE_COUNT + 1; LAST := 0; ELSE% SLINE ( 1..2 ) := "??"; FORCER AUTRE CHOSE QUE // EN CAS D
E LIGNE VIDE% GET LINE ( IFILE, SLINE, LAST ); LIRE UNE LIGNE% IF SLINE ( 1..2 ) = "://" THEN% NE S'OCQUPE QUE DES LIGNES COMMENTANT PA
R //% COL := 3; SE METTRE EN COL 3 POST "://" ELSE% COL := LAST + 1; POST FIN DE LIGNE POUR NEGLIGER LA LIGNE% END IF;
END IF; WHILE COL < LAST AND THEN ( SLINE (COL) = ' ' OR ELSE SLINE (COL) = ASCII.HT ) LOOP% PASSER LES BLANCS% COL := COL + 1;
END LOOP; IF COL < LAST THEN% IF SLINE (COL) = ' ' AND THEN COL < LAST AND THEN SLINE (COL+1) = ' ' THEN% COL := LAST + 1; ELSE%
LIGNE NON VIDE% SOURCELINE := MAKE ( DN_SOURCELINE ); CREER UN NOEUD LIGNE SOURCE% DI ( XD_NUMBER, SOURCELINE, LINE_COUNT );
Y METTRE LE NUMERO DE LIGNE% LIST ( SOURCELINE, (TREE_NIL, TREE_NIL) ); INITIALISER LA XD_ERROR_LIST% SOURCE_LIST := APPEND ( SOURCE_LIST,
SOURCELINE ); METTRE LA LIGNE EN FILE% EXIT; END IF; END LOOP; F_COL := COL; TOKEN_LENGTH := 1;
0020 TOKEN_IS_NAME := FALSE; CASE SLINE (COL) IS% WHEN 'A'..'Z' | 'a'..'z' | '0'..'9' THEN% TOKEN_IS_NAME := TRUE; COL := COL + 1; WHILE CO
L < LAST AND THEN ( SLINE (COL) IN 'A'..'Z' | 'a'..'z' | '0'..'9' OR ELSE SLINE (COL) IN ' ' OR ELSE SLINE (COL) IN ' ' )
DO% LOOP% TOKEN_LENGTH := TOKEN_LENGTH + 1; COL := COL + 1; END LOOP; WHEN '!' THEN% PEUT ETRE :=
IF COL + 2 <= LAST AND THEN SLINE (COL+1) = '!' AND THEN SLINE (COL+2) = '!' THEN% OUI UN :=
LONGUEUR% COL := COL + 3; ELSE% COL := COL + 1; END IF; WHEN '!' THEN% PEUT ETRE :=
IF COL + 1 <= LAST AND THEN SLINE (COL+1) = '!' THEN% OUI '!' THEN% TOKEN_LENGTH := 3; ELSE%
ST AND THEN SLINE (COL+1) = '!' THEN% OUI '>' THEN% TOKEN_LENGTH := 2; ELSE% COL := COL + 2; ELSE% COL := COL + 1;
D IF; WHEN '!' THEN% COL := COL + 1; WHEN OTHERS => COL := COL + 1; END CASE; SOURCEPOS := MAKE_SOURCE_POSITI
--PROCEDURE PROCESS_IDL% PROCEDURE PROCESS_IDL IS% RULE_NODE := TREE; NODE_LIST := SEQ_TYPE := (TREE_NIL, TREE_NIL); PRIOR_F
COL := POSITIVE := 1; PRIOR_TOKEN_LENGTH := NATURAL := 0; TYPE ATTR_TYPE := IS (NORMAL, SEQ);
0030 NG ) IS% SYMBOL := TREE := STORE_SYM ( NODE_NAME ); STOCKER/RETIRER LE NOM DE NOEUD REGLE (NOM EN PARTIE GAUCHE AVANT LE => ) R_LIST
:= SEQ_TYPE := LIST (SYMBOL); LISTE-CONTENANT LE NOEUD REGLE ASSOCIEES AU SYMBOLE (REFERENCE COMME GAUCHE DE REGLE OU TYPEAGE) BEGIN%
IF IS_EMPTY (R_LIST) THEN% LA LISTE EST VIDE, C'EST LA PREMIERE DEFINITION DE REGLE/CLASSE% RULE_NODE := MAKE ( DN_CLASS_NODE );
FABRIQUER UN NOEUD POUR LA REGLE/CLASSE% DI ( XD_SYMPRE, RULE_NODE, SYMBOL ); METTRE LE SYMBOLE DU NOM DE REGLE (NOM GAUCHE) DA
NS LE XD_NAME DU NOEUD REGLE% LIST ( RULE_NODE, (TREE_NIL, TREE_NIL) ); INITIALISER VIDE LA LISTE DES ELEMENTS DU CTE DROIT% IF
CONTEXT = IN_NODE THEN% DANS UN NOEUD REGLE SIMPLE% DB ( XD_IS_CLASS, RULE_NODE, FALSE ); MARQUER UNE REGLE DE STRUCTURE D'ATTRIBUT
S% ELSE% DANS UNE CLASSE% DB ( XD_IS_CLASS, RULE_NODE, TRUE ); MARQUER UNE REGLE DE DEFINITION DE CLASSE% END IF;
D ( LX_SRCSPOS, RULE_NODE, SOURCEPOS ); METTRE LA POSITION SOURCE-DONNEE PAR GET_TOKEN% D ( XD_PARENT, RULE_NODE, TREE_VOID );
LIST ( SYMBOL, APPEND ( (TREE_NIL, TREE_NIL), RULE_NODE ); METTRE LE NOEUD-REGLE COMME ELEMENT-UNIQUE DE LISTE DU SYMBOLE DE NOM GAUCHE
0040 (XD_DEFLIST) NODE_LIST := APPEND (NODE_LIST, RULE_NODE); METTRE EN LISTE LA REGLE% ELSE% UNE REGLE AVEC MEME PARTIE
GAUCHE A-DEJ ETE VUE% DECLARE DEF: TREE := HEAD (R_LIST); PRENDRE LE NOEUD DE TETE, LE NOEUD REGLE% BEGIN% IF DEF.TY = DN_
CLASS_NODE THEN% VERIFIER QUE C'EST UNE REGLE% RULE_NODE := DEF; PRENDRE CE NOEUD COMME COURANT% IF CONTEXT = IN_CLASS THEN%
ON A UN NOM CITE EN CLASSE (PEUT AVOIR ETE TROUVE AUPARAVANT CITE-COMME TYPEAGE EN DROITE DE REGLE D'ATTRIBUTION) DB ( XD_IS_CLASS, RULE_N
ODE, TRUE ); MARQUER COMME CLASSE% END IF; ELSE% ANOMALIE LE NOEUD N'EST PAS UNE REGLE% ERROR (SOURCEPOS, "HOLA !" & NODE_N
AME & " N'EST PAS UN NOM EN PARTIE GAUCHE (REGLE) !" ); END IF; END IF;
--PROCEDURE MAKE_ATTR% PROCEDURE MAKE_ATTR ( ATTR_NAME: STRING;
ATTR_TYPE: ATTR_TYPE; NOM_TYPE: STRING ) IS% SYMBOL := TREE := STORE_SYM ( ATTR_NAME ); STOCKER/RETIRER LE SYMBOLE (SYMPRE)-DE L'ATTRIBUT
A_LIST := SEQ_TYPE := LIST (SYMBOL); LISTE-CONTENANT LA DEFINITION DE L'ATTRIBUT% TYPEAGE := STORE_SYM (NOM_TYPE); SI
OCKER/REPRENDRE LE SYMBOLE DU TYPE DE L'ATTRIBUT (UN NOM DE REGLE OU DE CLASSE) ATTR := TREE; BEGIN% IF IS_EMPTY (A_LIST) THEN%
0050 -- AUCUNE APPARITION DE CET ATTRIBUT% ATTR_COUNT := ATTR_COUNT + 1; ATTRIBUT DE PLUS% ATTR := MAKE ( DN_ATTR );
CREER UN NOEUD DE TYPE ATTRIBUT/TERMINAL% D ( XD_SYMPRE, ATTR, SYMBOL ); POINTER LE SYMBOLE DANS LE CHAMP XD_SYMPRE DU TERMINAL%
CASE ATTR_TYPE IS% SUIVANT QUE L'ON A UN ATTRIBUT SIMPLE OU UNE SEQUENCE% WHEN NORMAL => UN ATTRIBUT SIMPLE% DI ( XD_ATTR
_ID, ATTR, ATTR_COUNT ); PORTER LE N D'ATTRIBUT EN POSITIF% WHEN OTHERS => UN ATTRIBUT SEQUENCE, PORTER LE N D'ATTRIBUT EN
NEGATIF% DI ( XD_ATTR_ID, ATTR, -ATTR_COUNT ); END CASE; D ( XD_ATTR_TYPE, ATTR, TYPEAGE ); METTRE LE TYPEAGE% LIST ( SY
MBOL, APPEND ( (TREE_NIL, TREE_NIL), ATTR ); PORTER L'ATTRIBUT CRE EN LISTE DANS LE XD_DEFLIST DU SYMPRE% ELSE% IL Y A DEJ UNE
APPARITION% ATTR := HEAD (A_LIST); PRENDRE LA DEFINITION EXISTANTE% END IF; IF ATTR.TY = DN_ATTR THEN% CE
DOIT ETRE UN ATTRIBUT/TERMINAL% CASE ATTR_TYPE IS% WHEN NORMAL => ATTRIBUT SIMPLE (NON SEQUENCE)% IF DI ( XD_ATTR_ID, ATTR ) < 0
THEN% IL Y A UNE ANOMALIE (LE N D'ATTRIBUT DOIT ETRE POSITIF-DANS CE CAS)% ERROR (SOURCEPOS, "ATTR IS SEQ & ATTR NAME" & " ELSE IF TYPEAGE /
= D (XD_ATTR_TYPE, ATTR) THEN% ANOMALIE SI LE TYPEAGE NE CORRESPOND-PAS CE QUI EST DANS L'ATTRIBUT (VOIR LA LIGNE (**) UN PEU-PLUS HAUT)% D
ECLARE% OLD_TYPEAGE := TREE := D ( XD_ATTR_TYPE, ATTR ); BEGIN% ERROR (SOURCEPOS, "VALUE OF " & ATTR_NAME & " IS " & PRINT_NAME ( TYPEAGE
) & " " & PAGE_IDX IMAGE ( TYPEAGE.PG ) & " & LINE_IDX IMAGE ( TYPEAGE.LN ) & " & NODE_NAME IMAGE ( TYPEAGE.TY ) & " ) CALLED " & PRINT_NA
ME ( TYPEAGE ) & " , NOT " & PRINT_NAME ( OLD_TYPEAGE ) & " & PAGE_IDX IMAGE ( OLD_TYPEAGE.PG ) & " & LINE_IDX IMAGE ( OLD_TYPEAGE.LN ) & " &
" & NODE_NAME IMAGE ( OLD_TYPEAGE.TY ) & " ) CALLED " & PRINT_NAME ( OLD_TYPEAGE ) & " ) END; END IF; WHEN OTHERS => ATTRIBUT S
EQUENCE% IF DI ( XD_ATTR_ID, ATTR ) = 0 THEN% ANOMALIE SI LE N D'ATTRIBUT EST POSITIF (IL DOIT ETRE NEGATIF POUR UNE SEQUENCE)% ERROR (SOUR
CEPOS, "ATTR IS NOT SEQ" & ATTR_NAME ); ELSE% IF TYPEAGE / = D (XD_ATTR_TYPE, ATTR) THEN% SI LE TYPEAGE DE SEQUENCE DIFFERE DE L'ANTERIEUR% DECL
ARE% TEMP := TREE := ATTR; GARE L'ATTRIBUT ANCIENNEMENT TYPE% BEGIN% ATTR := MAKE ( DN_ATTR ); REFAIRE UN TERMINAL QUI E
ST PRIS COMME ATTRIBUT% D ( XD_SYMPRE, ATTR, D (XD_SYMPRE, TEMP ); DANS LE SYMPRE DU NOUVEAU, RECUPERER LE SYMPRE-DE L'ANCIEN% DI (
XD_ATTR_ID, ATTR, DI (XD_ATTR_ID, TEMP ); REPORTER AUSSI LE N D'ATTRIBUT% D ( XD_ATTR_TYPE, ATTR, TYPEAGE ); METTRE LE TYPEAGE D
ANS CE NOUVEAU NOEUD ATTRIBUT QUI DIFFERE PAR LE TYPEAGE% END; END IF; DECLARE ASEQ% SEQ_TYPE := LIST ( RULE_NODE );
0070 -- PRENDRE LA XD_LIST DU NOEUD REGLE-EN COURS% BEGIN% LIST ( RULE_NODE, APPEND ( ASEQ, ATTR ); AJOUTER AU NOEUD REGLE EN-COURS LA
LISTE DES ATTRIBUTS AUGMENTEE% END; ELSE% LA DEFINITION (TROUVEE) N'EST PAS UN TERMINAL% ERROR (SOURCEPOS, "NOT DEFINED AS AN ATTRIBUTE " & ATTR_NAME );
END IF; END MAKE_ATTR;
--PROCEDURE MAKE_MEMBER% PROCEDURE MAKE_MEMBER ( MEMBER_NAME: STRING ) IS% SYMBOL := TREE := STORE_SYM ( MEMB
ER_NAME ); STOCKER/REPRENDRE LE SYMPRE CORRESPONDANT AU NOM D'ELEMENT DE CLASSE% MEMBER := TREE := MAKE ( DN_MEMBER ); FABRIQUER UN
NOM TERMINAL-POUR UN ELEMENT APPARTENANT A UNE CLASSE% M_LIST := SEQ_TYPE := LIST (SYMBOL); LA LISTE CONTENANT LA REGLE DEFINISSANT LE SY
MBOLE% BEGIN% D (XD_SYMPRE, MEMBER, SYMBOL ); METTRE LE SYMPRE DANS LE CHAMP XD_SYMPRE DU MEMBRE DE CLASSE% D ( LX_SRCSPOS, MEMB
ER, SOURCEPOS ); METTRE LA POSITION SOURCE-DU MEMBRE-CRE PAR GET_TOKEN% LIST ( RULE_NODE, APPEND ( LIST (RULE_NODE), MEMBER );
-- AJOUTER LE MEMBRE LA LISTE DES MEMBRES DU NOEUD% END;
--BEGIN% LAST := 0; COL := 1; GET_TOKEN; LOOP% EXIT WHEN END_OF_FILE (IFILE) OR ELSE SLINE (F_COL..F_COL + TOKEN_LENGTH
= "end"); FINIR-AVEC LE FICHIER OU LE LEXEME %%% QUI INDIQUE LA FIN% IF TOKEN_IS_NAME THEN% LEXEME IDENTIFICATEUR%
PRIOR_F_COL := F_COL; GARDER SA POSITION% PRIOR_TOKEN_LENGTH := TOKEN_LENGTH; ET SA LONGUEUR% GET_TOKEN;
-- ET PASSER AU SUIVANT (QUI-VA PERMETTRE DE SAVOIR CE QUE L'ON VA FAIRE)% ELSE% SLINE (F_COL..F_COL + TOKEN_LENGTH - 1) = " " THEN% INDO
UE UNE REGLE DEFINISSANT DES ATTRIBUTS% CONTEXT := IN_NODE; GARDER UNE TRACE DE CE FAIT : DEFINITION D'UNE ASSOCIATION D'ATTRIBUTS%
MAKE_RULE_OR_CLASS_NODE ( SLINE ( PRIOR_F_COL..PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) ); TENTER LA CREATION D'UN NOEUD REGLE-(OU LE RAMENER S
IL-EXISTE DEJ)% GET_TOKEN; ALLER-CHECHER LE PREMIER-NOM D'ATTRIBUT OU LE %%% ELSE% SLINE (F_COL..F_COL + TOKEN_LENGTH - 1) = " " THEN%
L'EXISTE-DEJ% LEXEME MARQUANT UNE DEFINITION DE CLASSE% CONTEXT := IN_CLASS; GARDER UNE TRACE DE CE FAIT : DEFINITION D'UNE CLAS
SE% MAKE_RULE_OR_CLASS_NODE ( SLINE ( PRIOR_F_COL..PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) ); TENTER LA CREATION D'UN NOEUD REGLE-(OU LE RAMENE
R S'IL-EXISTE DEJ)% GET_TOKEN; ALLER-CHECHER UN COMPOSANT DE CLASSE% WHILE SLINE (F_COL..F_COL + TOKEN_LENGTH - 1) / = " "
LOOP% JUSQU' LA FIN DE LA DEFINITION DE CLASSE% IF TOKEN IS NAME THEN% SI L'ON A UN NOM (PAS UNE " " SEPARATEUR)% MAKE_MEMBER ( SLINE
(F_COL..F_COL + TOKEN_LENGTH - 1 ) ); CREER UN MEMBRE DE CLASSE% END IF; GET_TOKEN; AVANCER AU LEXEME SUIVANT% END LOOP;
ELSE% SLINE (F_COL..F_COL + TOKEN_LENGTH - 1) = " " THEN% SEPARATEUR DU TYPEAGE% GET_TOKEN; AMENER UN SEQ OU LE NOM DU TY
PE% IF SLINE (F_COL..F_COL + TOKEN_LENGTH - 1) = "Seq" THEN% C'EST UN SEQ% GET_TOKEN; PRENDRE LE OF OU LE NOM DE TYPE% IF SLINE
(F_COL..F_COL + TOKEN_LENGTH - 1) = "Of" THEN% ON A LE OF% GET_TOKEN; PRENDRE LE NOM DE TYPE% END IF; MAKE_ATTR ( SLINE ( PRIOR_F
COL..PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) ); SEQ, AJOUTER UN ATTRIBUT SEQUENCE% SLINE (F_COL..F_COL + TOKEN_LENGTH - 1 ) AVEC SON TYPEAGE%
ELSE% SLINE ( PRIOR_F_COL..PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ) / = "Lx comments" THEN% SI CE N'EST PAS UN ATTRIBUT LX COMMENTS% MAKE
ATTR ( SLINE ( PRIOR_F_COL..PRIOR_F_COL + PRIOR_TOKEN_LENGTH - 1 ), NORMAL, AJOUTER UN ATTRIBUT SIMPLE (NON SEQUENCE)% SLINE (F_COL..F_COL
```

```

IS_CLASS, NODE ) THEN-----| UN NOEUD CLASSE--DECLARE THE_NAME: CONSTANT STRING:= PRINT_NAME( D ( XD_SYMPREP, -NODE ) );--BEGIN IF THE_NAME
/="NON_DIANA"--AND THEN THE_NAME /="ALL_SOURCE"--AND THEN THE_NAME /="TYPE_SPEC"--AND THEN THE_NAME /="STANDARD_IDL"--THEN--PUT
LINE ( "**** PARTITION INATTENDUE = " & THE_NAME );--END IF;--PUT( THE_NAME );--IF NOT IS_CLASS THEN--PUT( CFILE, THE_NAME );--END IF;--EN
D;--ELSE-----| UN NOEUD DE REGLE D'ATTRIBUTION--PUT( "...");--END IF;--ELSE-----| PAS CLASSE DE BASE--CLASS_P
ATH(-PARENT, IS_CLASS );-----| REMONTER VERS LA CLASSE DE BASE--PUT( " > " & PRINT_NAME( D( XD_SYMPREP, NODE ) ) );--IF NOT IS_CLASS
THEN--PUT( CFILE, " > " & PRINT_NAME( D ( XD_SYMPREP, NODE ) ) );-----| REPETER--END IF;--END IF;--END CLASS_PATH;--BEGIN CREAT
E ( NFILE, -OUT_FILE, -NOM_TEXTE-& ".nodes.txt" );-----| FICHER INFORMATION TEXTE-DES NOEUDS--CREATE ( CFILE, -OUT_FILE, -NOM_TEXTE-& ".class.txt"
);-----| FICHER INFORMATION TEXTE-HIERARCHIE DES CLASSES--SET_OUTPUT ( NFILE );--PUT_LINE ( "----- ARBORESCENCE IDL -----");--WHILE-NOT I
0130 S_EMPTY ( NODE_LIST ) LOOP--POP( NODE_LIST, RULE_NODE );--DECLARE IS_A_CLASS: BOOLEAN:= DB ( XD_IS_CLASS, RULE_NODE )
;--BEGIN--DECLARE-RULE_CLASS_NAME: CONSTANT STRING:= PRINT_NAME ( D (-XD_SYMPREP, RULE_NODE ) );--BEGIN IF IS_A_CLASS THEN-----|
DEFINIT UNE CLASSE--PUT ( "f" & RULE_CLASS_NAME-& "}" );--ELSE--PUT ( RULE_CLASS_NAME );--END IF;--END;--PUT ( ASCII.HT & "
PATH " );--CLASS_PATH ( RULE_NODE, IS_CLASS=> DB ( XD_IS_CLASS, RULE_NODE ) );--IF NOT IS_A_CLASS THEN-----| PAS UNE CLASSE--NEW_LIN
E ( CFILE );--END IF;--ITEM_LIST := LIST ( RULE_NODE );-----| LISTE-DES ATTRIBUTS OU DES MEMBRES--IF NOT IS_EMPTY ( ITEM_LIST
) THEN--IF IS_A_CLASS THEN-----| DEFINIT UNE CLASSE--PUT ( " := " );--END IF;--NEW_LINE;--END;--WHILE NOT I
S_EMPTY ( ITEM_LIST ) LOOP--POP ( ITEM_LIST, ITEM );--IF ITEM.TY = DN_ATTR THEN-----| ATTRIBUT--PUT ( ASCII.HT & "> " & PRINT_NAME
( D(-XD_SYMPREP, ITEM ) & ASCII.HT & ":" );--IF DI ( XD_ATTR_ID, -ITEM ) < 0 THEN-----| ATTRIBUT SEQUENCE--PUT ( "SEQ OF " );--END IF;--PUT_LINE
( PRINT_NAME ( D ( XD_ATTR_TYPE, ITEM ) ) );-----| TYPAGE--ELSE-----| MEMBRE DE CLASSE--PUT ( ASCII.HT & PRINT_NAME ( -D ( XD_SYMPREP, I
TEM ) ) );-----| NOM DU MEMBRE--DEFLIST := LIST ( D(-XD_SYMPREP, ITEM );--IF IS_EMPTY ( DEFLIST ) OR ELSE HEAD ( DEFLIST ).TY /= DN_CLASS_NODE THEN
0140 --PUT ( " ?????" );--END IF;--NEW_LINE;--END IF;--END-LOOP;--NEW_LINE;--END LOOP;--SET_OUTPUT ( STANDARD_OUTPUT );--CLOSE
(-NFILE );--CLOSE( CFILE );--EXCEPTION WHEN OTHERS =>--CLOSE ( NFILE );--CLOSE ( CFILE );--PUT_LINE ( "ERREUR A L IMPRESSION
" );--END PRINT_IDL;--BEGIN OPEN ( IFILE, IN_FILE, ".../idl/" & -NOM_TEXTE-& ".idl" );-----| FICHER SOURCE IDL--PUT_LINE ( "LE FICHER : " &
NOM_TEXTE & ".idl EST OUVERT " );--CREATE_IDL_TREE_FILE ( NOM_TEXTE & ".lar" );-----| FICHER D'ARBRE-IDL--PUT_LINE ( "LE FICHER : " & NOM_TEX
TE & ".lar EST CREE" );--SOURCE_LIST := (TREE_NIL, TREE_NIL);--PUT_LINE ( "PROCESS IDL ..." );--PROCESS_IDL;--PUT_LINE ( "OK" );--LIST ( TREE_ROOT,
-SOURCE_LIST );--CLOSE (-IFILE );--CHECK_IDL;--PRINT_IDL;--CLOSE_IDL_TREE_FILE;--EXCEPTION WHEN NAME_ERROR =>--PUT_LINE ( "LE FICHER DESCR
PTION : " & ".../idl/" & -NOM_TEXTE-& ".idl EST INTROUVABLE" );--END IDL_READ;
-----|END IDL_READ;

```